

Cloud-Native DevOps Platform

PART-1

SEETHA LAKSHMI K A P

23-06-2026

Application Development, Dockerization & Infrastructure Provisioning

1)Objective

Develop the frontend and backend applications, containerize them using Docker, organize the project structure, and provision AWS infrastructure using Terraform.

2) Backend Development

2.1) Files Created

```
backend/  
    app.py  
    database.py  
    requirements.txt  
    Dockerfile  
    test-app.py
```

2.2) app.py

Purpose

Main Flask application.

Implemented

- REST APIs
- Health endpoint
- Database operations
- File upload API
- JSON responses

Reason

Acts as the backend service that communicates with the frontend, Amazon RDS, and Amazon S3.

2.3) database.py

Purpose

Handles MySQL database connection.

Contains

- Database host
- Username
- Password
- Connection object

Reason

Keeps database logic separate from application logic.

2.4) requirements.txt

Contains

Flask
Gunicorn
mysql-connector-python
boto3
prometheus_client

Command

```
pip install -r requirements.txt
```

Purpose

Installs all backend dependencies.

2.5) test-app.py

Purpose

Used for backend testing.

Reason

Executed during GitHub Actions workflow before deployment.

2.6) Backend Docker

Dockerfile

Purpose

Containerizes the Flask application.

Steps Performed

- Python base image
- Copy source code
- Install dependencies
- Expose port
- Run Gunicorn

Commands

```
docker build -t backend .
```

```
docker run -p 5000:5000 backend
```

3) Frontend Development

3.1) Files Created

```
frontend/  
    index.html  
    style.css  
    script.js  
    nginx.conf  
    Dockerfile
```

3.2) index.html

Purpose

Creates the user interface.

Contains

- Upload button
- Forms
- Response area

3.3) style.css

Purpose

Styles the webpage.

3.4) script.js

Purpose

Handles frontend logic.

Functions

- API calls
- Upload requests
- Display responses

3.5) nginx.conf

Purpose

Configures Nginx.

Used For

- Serve static website
- Reverse proxy (if needed)

3.6) Frontend Docker

[Dockerfile](#)

Purpose

Creates frontend Docker image.

Steps

- Nginx base image
- Copy HTML
- Copy CSS
- Copy JS
- Copy nginx.conf

Commands:

```
docker build -t frontend .  
docker run -p 80:80 frontend
```

4) Terraform Infrastructure

Purpose

Terraform is used to provision all AWS resources automatically using Infrastructure as Code (IaC).

4.1) provider.tf

Purpose

Configures AWS provider.

Contains

- AWS Region
- Provider configuration

Command

```
terraform init
```

4.2) variables.tf

Purpose

Stores reusable variables.

Examples

- Cluster name
- Region
- Instance type

Reason

Avoids hardcoding values.

4.3) terraform.tfvars

Purpose

Stores actual variable values.

4.4) network.tf

Creates

- VPC
- Public Subnets
- Private Subnets
- Internet Gateway
- NAT Gateway
- Route Tables

Reason

Provides networking for EKS and RDS.

4.5) security.tf

Creates

- Security Groups

Allows

- HTTP
- HTTPS
- MySQL
- Kubernetes communication

4.6) eks.tf

Creates

- Amazon EKS Cluster
- Managed Node Group

Reason

Runs Kubernetes workloads.

4.7) ecr.tf

Creates

- Amazon ECR Repository for both backend and frontend

Reason

Stores Docker images.

4.8) rds.tf**Creates**

- Amazon RDS MySQL

Reason

Stores application data.

4.9) s3.tf**Creates**

- Amazon S3 Bucket

Used For

- File uploads
- Lambda trigger

4.10) github_oidc.tf**Creates**

GitHub OIDC Identity Provider.

Reason

Allows GitHub Actions to access AWS securely without AWS access keys.

4.11) lambda_iam.tf**Creates**

IAM Role for Lambda.

Permissions

- S3

- CloudWatch Logs

4.12) alb_controller.tf

Creates

AWS Load Balancer Controller IAM Role.

Reason

Automatically creates Application Load Balancers for Kubernetes Ingress.

4.13) external_secrets_irsa.tf

Creates

IAM Role for External Secrets.

Reason

Allows Kubernetes pods to read AWS Secrets Manager securely.

4.14) secrets.tf

Creates

AWS Secrets Manager Secret.

Stores

- Database credentials

4.15) s3_notification.tf

Purpose

Configures S3 Event Notifications.

Reason

Triggers Lambda whenever a file is uploaded.

4.16) loki_s3.tf

Purpose

Provides S3 storage for Loki logs.

4.17) s3_node_policy.tf

Purpose

Allows Kubernetes worker nodes to access S3.

outputs.tf

Displays

- Cluster Name
- Cluster Endpoint
- VPC ID
- ECR URL

5) Infrastructure Deployment

Commands used:

```
terraform init
terraform plan
terraform apply -auto-approve
```

6) Output Screenshots

```
PS C:\Users\user\Downloads\task-5\terraform> terraform apply
kubernetes_service_account.alb_controller: Creation complete after 2s [id=kube-system/aws-load-balancer-controller]
helm_release.aws_load_balancer_controller: Creating...
helm_release.aws_load_balancer_controller: Still creating... [00m10s elapsed]
helm_release.aws_load_balancer_controller: Still creating... [00m20s elapsed]
helm_release.aws_load_balancer_controller: Still creating... [00m30s elapsed]
helm_release.aws_load_balancer_controller: Still creating... [00m40s elapsed]
helm_release.aws_load_balancer_controller: Creation complete after 45s [id=aws-load-balancer-controller]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

backend_ecr = "622488711452.dkr.ecr.us-west-1.amazonaws.com/task5-backend"
frontend_ecr = "622488711452.dkr.ecr.us-west-1.amazonaws.com/task5-frontend"
github_role_arn = "arn:aws:iam::622488711452:role/github-actions-role"
lambda_name = "task5-s3-email"
s3_bucket = "task5-uploads"
```

Amazon Elastic Kubernetes Service > Clusters

Clusters (1) info

Filter clusters

Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created
task5-eks	Active	1.31 Upgrade now	Extended support until November 26, 2026	Extended support	June 25, 2026, 12:27 (UTC+05:30)

VPC > Your VPCs

VPC dashboard

Virtual private cloud

Your VPCs

VPCs VPC encryption controls

Your VPCs (2) info

Find VPCs by attribute or tag

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv
task5-vpc	vpc-0bc385fcff5ede95b	Available	-	-	Off	10
-	vpc-0be40215c95d16a82	Available	-	-	Off	17

Select a VPC above

us-west-1.console.aws.amazon.com/ec2/home?region=us-west-1#Instances:

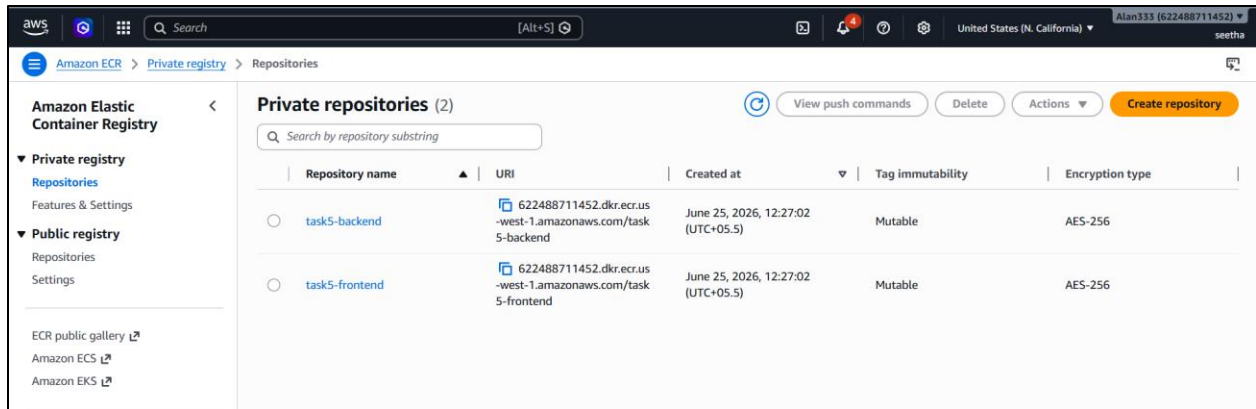
EC2 > Instances

Instances (4) info

Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	F
default	i-0dcdf02842470639d	Running	t3.medium	3/3 checks passed	us-west-1a	-	-
default	i-0b1c2e0885fcc3f11	Running	t3.medium	3/3 checks passed	us-west-1a	-	-
default	i-0a9517a73c27b9e48	Running	t3.medium	3/3 checks passed	us-west-1c	-	-
default	i-0cd5e1b05ba8ab334	Running	t3.medium	3/3 checks passed	us-west-1c	-	-

Select an instance



7) EKS Cluster Connectivity Verification

After provisioning the EKS cluster, the local kubeconfig was updated and cluster connectivity was verified using the following commands:

```
aws eks update-kubeconfig --region ap-south-1 --name <cluster-name>
kubectl cluster-info
kubectl get nodes
kubectl get pods -A
```

8) Outcome

- Backend microservice developed
- Frontend application completed
- Project pushed to GitHub
- Complete AWS infrastructure defined using Terraform
- Ready for Kubernetes deployment in the next phase